

FOOD DELIVERY TIME PREDICTION

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

BY

S.Jithamanyu

24501A05M0

Y.Vaishnavi

24501A05P6

S.Nidhi Kumari

24501A05L5

T.Sruthi

24501A05N6



PRASAD V POTLURI SIDDHARTHA INSTITUTE OF TECHNOLOGY

(Permanently affiliated to JNTU Kakinada, Approved by AICTE)

(An NBA & NAAC A+ accredited and ISO 21001:2018 Certified Institution)

Kanuru, Vijayawada – 520007

(2026-27)

PRASAD V POTLURI

SIDDHARTHA INSTITUTE OF TECHNOLOGY

(Permanently affiliated to JNTU Kakinada, Approved by AICTE)

(An NBA & NAAC accredited and ISO 21001:2018 certified
institution) **Kanuru, Vijayawada – 520007**



CERTIFICATE

This is to certify that the project report title “**Food Delivery TimePrediction**” is the bonafied work of **S.Jithamanyu (24501A05M0)**, **Y.Vaishnavi (24501A05P6)**, **S.Nidhi Kumari (24501A05L5)**, **T.Sruthi (24501A05N6)** in partial fulfilment of completing the Academic project in Data Mining during the academic year 2026-27.

Signature of the course coordinator

Signature of the HOD

INDEX

S.No.	Content	Page No.s
1	Abstract	5
2	Introduction	6
3	Dataset Description	7-8
4	Project Design	9-10
5	Data Preprocessing	11-12
6	Exploratory Data Analysis	13-14
7	Model Training	15-16
8	Model Evaluation	17-18
9	Model Selection	19
10	Implementation	20-21
11	conclusion	22
12	References	23

LIST OF FIGURES

Figure No.	Caption
Figure 4.1	<i>Proposed Project Design</i>
Figure 6.1	<i>Distribution of Delivery Time</i>
Figure 6.2	<i>Distance vs Delivery Time</i>
Figure 6.3	<i>Traffic Density vs Delivery Time</i>
Figure 6.4	<i>Correlation Heatmap of Numeric Variables</i>
Figure 8.1	<i>Performance Comparison of Models</i>
Figure 8.2	<i>RMSE Comparison of Models</i>
Figure 8.3	<i>Comparison of MAE, RMSE and R^2 (%) across Models</i>
Figure 9.1	<i>Actual vs Predicted Delivery Time (Gradient Boosting)</i>
Figure 10.1-10.6	<i>Implementation</i>

1. Abstract

The Food Delivery Time Prediction project is a **machine learning-based system** developed to estimate the time required for delivering food orders. Delivery time can be influenced by various factors such as **delivery distance, delivery person details, weather conditions, traffic density, city type, vehicle type, order type, vehicle condition, multiple deliveries, and festival season**. Considering these factors helps in understanding the conditions that affect food delivery time.

The project uses the **Zomato Delivery Dataset**, which contains delivery-related information required for developing the prediction system. The data is processed using appropriate preprocessing and **feature engineering** techniques to prepare it for machine learning. The target variable used for prediction is **Time_taken (min)**.

Different regression algorithms, including **Linear Regression, KNN Regression, Decision Tree Regression, and Gradient Boosting Regression**, are trained and evaluated using standard regression metrics such as **MAE, RMSE, and R² Score**. The trained model is integrated into a **web-based application**, where users can enter delivery-related details and obtain an **estimated delivery time**. Overall, the project demonstrates the practical application of **machine learning and web technologies** for food delivery time prediction.

2. Introduction

Food delivery services have become an important part of everyday life, where customers expect their orders to arrive within a reasonable and predictable time. However, the actual delivery time can vary depending on several factors, making it difficult to provide an accurate estimate for each order. Factors such as **delivery distance, traffic density, weather conditions, delivery person experience, vehicle condition, city type, multiple deliveries, and festival season** can have an impact on the time taken to complete a delivery.

Traditional methods of estimating delivery time may not effectively consider all these factors together. As a result, there is a need for a system that can analyze previous delivery data and identify patterns related to delivery time. Machine learning can be used for this purpose by learning from historical delivery information and using the learned patterns to estimate the delivery time for new orders.

The proposed **Food Delivery Time Prediction** system addresses this problem by using delivery-related data and regression-based machine learning techniques. The system processes the available data, identifies relevant factors, trains suitable prediction models, and provides an estimated delivery time based on the details of a new order. This approach aims to provide a simple, data-driven method for estimating food delivery time.

3. Dataset Description

The dataset is the *Zomato Delivery Operations Analytics Dataset* from Kaggle (saurabhbhadole/zomato-delivery-operations-analytics-dataset), loaded in Google Colab with kagglehub. The file used is `Zomato Dataset.csv`.

Property	Value
Rows	45,584
Columns	20
Target variable	Time_taken (min)
Mean delivery time	26.29 minutes
Median delivery time	26 minutes
Standard deviation	9.38 minutes
Duplicate rows in raw data	0

Columns in the raw dataset: ID, Delivery_person_ID, Delivery_person_Age, Delivery_person_Ratings, Restaurant_latitude, Restaurant_longitude, Delivery_location_latitude, Delivery_location_longitude, Order_Date, Time_Orderd, Time_Order_picked, Weather_conditions, Road_traffic_density, Vehicle_condition, Type_of_order, Type_of_vehicle, multiple_deliveries, Festival, City, Time_taken (min).

Missing values in the raw data

Column	Missing values
Delivery_person_Age	1,854
Delivery_person_Ratings	1,908
Time_Orderd	1,731
Weather_conditions	616
Road_traffic_density	601
multiple_deliveries	993
Festival	228
City	1,200

Descriptive statistics (mean, median, mode, standard deviation) were computed for the numeric columns. Delivery-person age ranges from 15 to 50 with a mean of about 29.6, and ratings average about 4.63. Some restaurant coordinates are zero or negative, which signals invalid location records. This is the reason the distance values are validated later.

4. Project Design

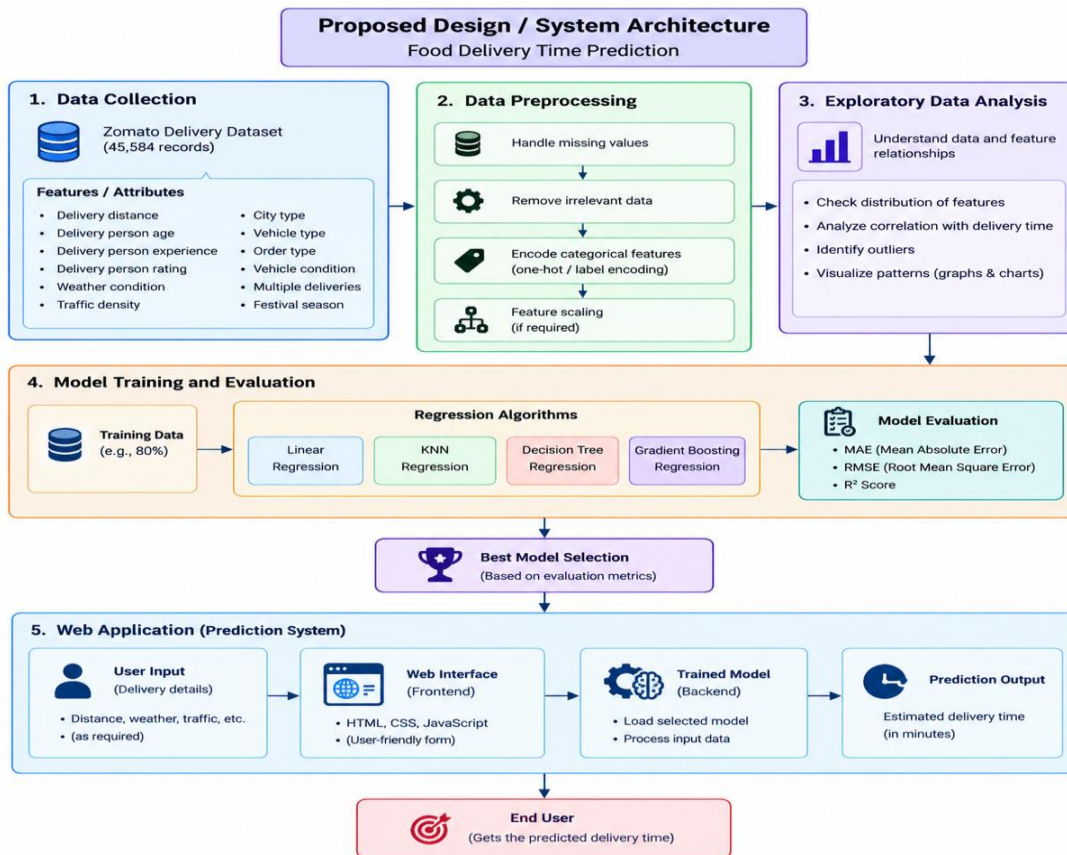


Figure 4.1 – Proposed Project Design

The workflow:

1. **Data collection:** download the dataset from Kaggle and load it into a DataFrame.
2. **Data understanding:** inspect shape, data types, statistics, missing values and duplicates.
3. **Data preprocessing:** drop identifier columns, engineer the distance feature, select features, convert types, impute missing values and remove duplicates.
4. **Exploratory data analysis:** visualise the target, distance, traffic density and correlations.
5. **Train-test split:** 80 : 20 split.
6. **Preprocessing pipeline:** one-hot encoding of categorical features inside a scikit-learn ColumnTransformer.

7. **Model training:** Linear Regression, KNN, Decision Tree and Gradient Boosting.
8. **Model evaluation:** MAE, RMSE and R^2 on the test set.
9. **Model selection and saving:** the final pipeline is saved with `joblib`.
10. **Web Interface:** The web interface provides a user-friendly form for entering the delivery information. It sends the entered values to the backend prediction system for processing.

5.Data Preprocessing

5.1 Removing Unnecessary Columns

The ID and Delivery_person_ID columns only identify rows and riders and carry no predictive information, so they were dropped. The dataset shape changed from $45,584 \times 20$ to $45,584 \times 18$.

5.2 Feature Engineering – Distance

The dataset gives coordinates rather than distance, so the straight-line distance between the restaurant and the delivery location was computed with the **Haversine formula**:

$$a = \sin^2(\Delta lat / 2) + \cos(lat1) \cdot \cos(lat2) \cdot \sin^2(\Delta lon / 2)$$

$$distance_km = 2 \cdot R \cdot \arcsin(\sqrt{a}), \quad R = 6371 \text{ km}$$

The raw distance feature contained impossible values caused by bad coordinates. Before cleaning it had a median of 9.26 km but a maximum of 19,692 km and a standard deviation of 1,099 km. To handle this, infinite values and any distance that was less than or equal to 0 km or greater than 300 km were set to missing and later imputed.

5.3 Attribute Selection

Eleven input features were selected to predict the target Time_taken (min).

Type	Features
Numeric (5)	Delivery_person_Age, Delivery_person_Ratings, distance_km, Vehicle_condition, multiple_deliveries
Categorical (6)	Weather_conditions, Road_traffic_density, Type_of_order, Type_of_vehicle, Festival, City

The raw latitude and longitude columns are replaced by distance_km. The date and time columns were not used.

5.4 Data Type Conversion and Handling Missing Values

Numeric columns were converted with `pd.to_numeric(errors='coerce')`, so invalid entries become missing values. Missing values were then filled as follows:

- **Numeric columns:** filled with the column **median**, which is robust to outliers.
- **Categorical columns:** filled with the column **mode** (most frequent category).

After imputation, no missing values remained in any model column.

5.5 Removing Duplicates

After feature selection, 10 duplicate rows were found and removed, reducing the dataset from 45,584 to **45,574 rows** with 12 columns (11 features and 1 target).

5.6 Encoding Categorical Features

Machine learning models need numeric inputs, so the six categorical features were converted with **One-Hot Encoding** (`OneHotEncoder(handle_unknown='ignore')`). Each category becomes its own 0/1 column. The `handle_unknown='ignore'` option stops the model from failing if an unseen category appears at prediction time. The numeric features are passed through unchanged. Encoding is part of a `ColumnTransformer`, which is included in every model pipeline so that training and prediction use identical preprocessing.

6. Exploratory Data Analysis

Four visualisations were produced from the cleaned data.

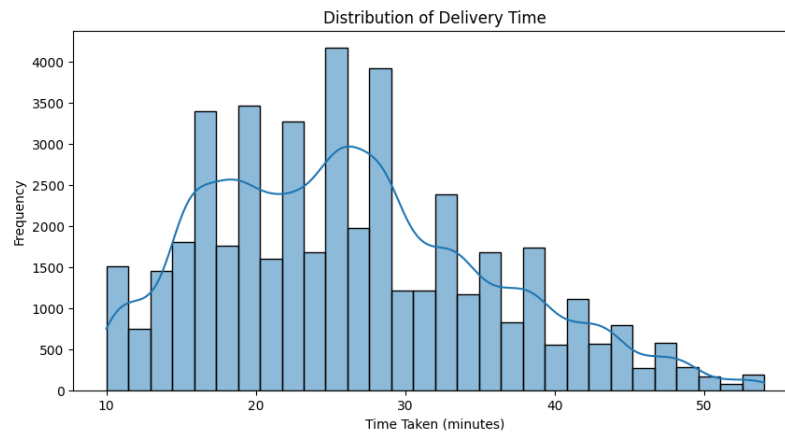


Figure 6.1 – Distribution of Delivery Time

The histogram with a density curve shows how delivery times are spread across the dataset.

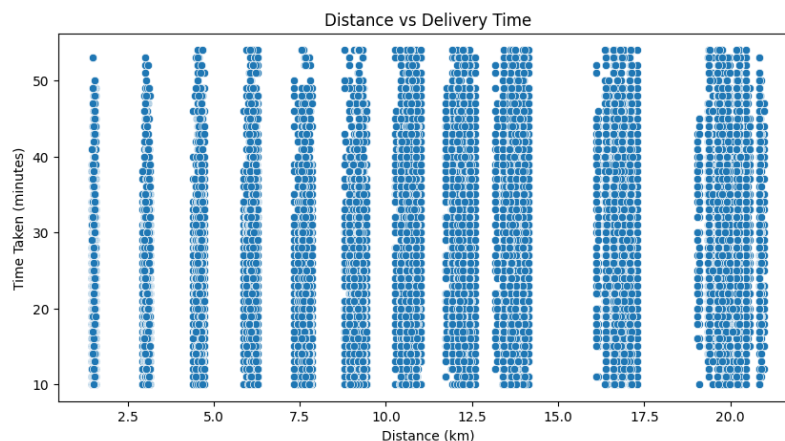


Figure 6.2 – Distance vs Delivery Time

Distances fall into distinct bands, and delivery times at every distance range from about 10 to 54 minutes. Distance alone therefore does not determine delivery time, which supports using the other features as well.

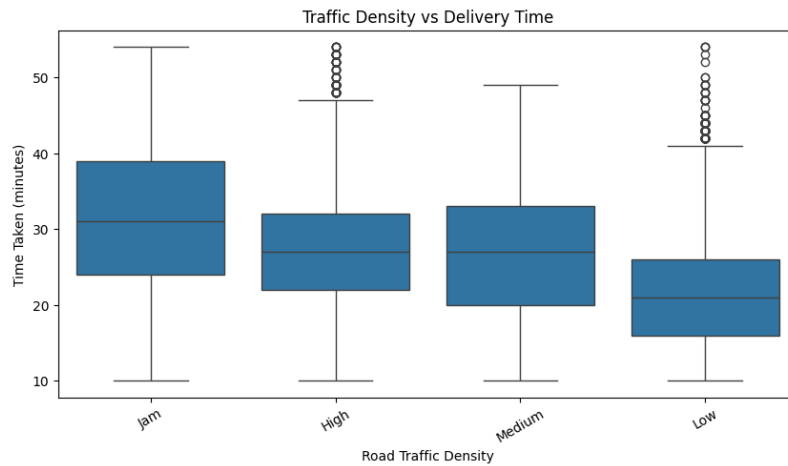


Figure 6.3– Traffic Density vs Delivery Time

Delivery time rises with congestion. Jam traffic has the highest median (about 31 minutes) and Low traffic the lowest (about 21 minutes), with High and Medium in between at roughly 27 minutes.

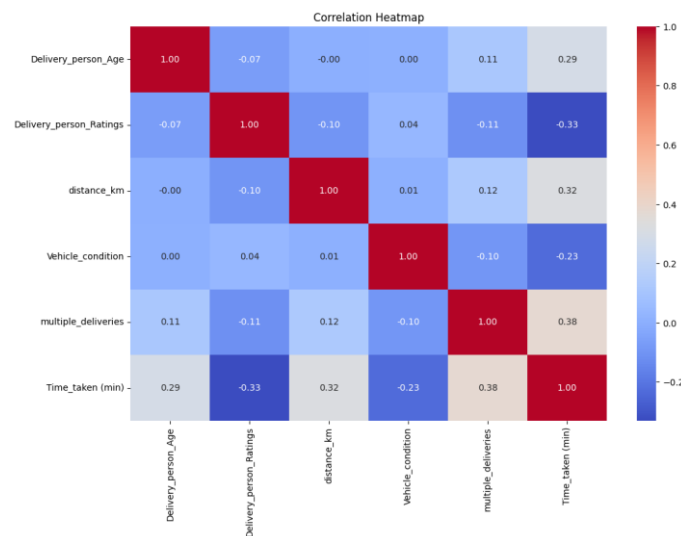


Figure 6.4– Correlation Heatmap of Numeric Variables

7. Model Training

The cleaned data (45,574 rows) was split into training and testing sets in an **80 : 20 ratio** with `random_state=42`, giving **36,459 training samples** and **9,115 testing samples**. The target variable is the delivery time in minutes. Each model was built as a scikit-learn `Pipeline` consisting of the preprocessing step followed by the regressor.

7.1 Linear Regression

Linear Regression assumes a linear relationship between the inputs and the target:

$$\hat{y} = \beta_0 + \beta_1 \cdot x_1 + \beta_2 \cdot x_2 + \dots + \beta_n \cdot x_n$$

where $\hat{y}$ is the predicted delivery time, x_i are the input features, β_i the learned coefficients and β_0 the intercept. It is simple and easy to interpret, but it cannot capture non-linear effects such as the combined influence of traffic and distance. It was trained with default parameters.

7.2 K-Nearest Neighbours Regression

KNN is a non-parametric algorithm. The prediction is the average target value of the K nearest training samples in feature space:

$$\hat{y} = (1/K) \cdot \sum y_i \text{ (over the } K \text{ nearest neighbours of } x)$$

The model used $K = 5$. Because KNN is distance-based, features on larger scales can dominate the neighbour search, and no scaling was applied in this project (see the limitations section).

7.3 Decision Tree Regression

A Decision Tree repeatedly splits the data on feature thresholds so that the samples in each branch have similar delivery times. The prediction for a sample is the average target value in the leaf it reaches. It can model non-linear relationships and feature interactions and does not need feature scaling. The tree depth was limited with `max_depth = 10` to control overfitting, with `random_state=42`.

7.4 Gradient Boosting Regression

Gradient Boosting is an ensemble method that builds decision trees one after another, where each new tree learns to correct the residual errors of the trees before it. The final prediction is the sum of the contributions of all trees:

$$\hat{y} = \sum f_k(x), \quad k = 1 \dots K$$

The model used **n_estimators = 100**, **learning_rate = 0.1** and **random_state=42**. The learning rate shrinks the contribution of each tree, which helps the ensemble generalise.

Hyperparameter summary

Model	Hyperparameters used
Linear Regression	Default
KNN Regression	n_neighbors = 5
Decision Tree Regression	max_depth = 10, random_state = 42
Gradient Boosting Regression	n_estimators = 100, learning_rate = 0.1, random_state = 42

The hyperparameters were set manually. Grid search and cross-validation were not applied in this version of the project.

8. Model Evaluation

All models were evaluated on the same 9,115 test samples with three metrics:

- **MAE (Mean Absolute Error):** the average absolute difference between predicted and actual delivery time, in minutes.
- **RMSE (Root Mean Squared Error):** the square root of the average squared error, in minutes. It penalises large errors more than MAE.
- **R² Score:** the proportion of variance in delivery time explained by the model (closer to 1 is better).

	Model	MAE	RMSE	R2	R2_Percentage
2	Decision Tree Regression	3.316044	4.214558	0.798539	79.853883
3	Gradient Boosting Regression	3.670534	4.609406	0.759022	75.902205
1	KNN Regression	4.367438	5.572367	0.647818	64.781828
0	Linear Regression	4.864614	6.115559	0.575811	57.581077

Figure 8.1 –Performance Comparison of Models

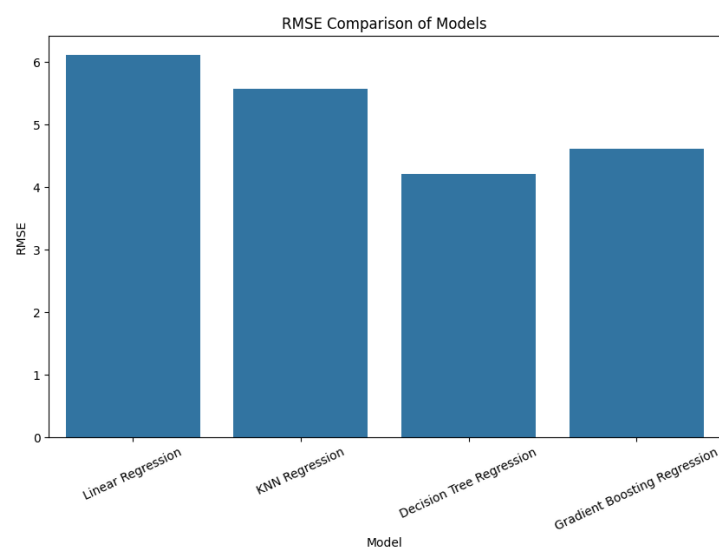


Figure 8.2 – RMSE Comparison of Models

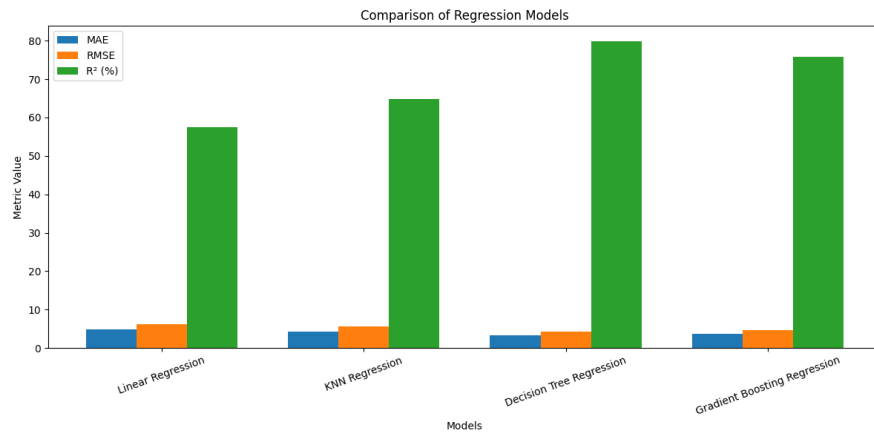


Figure 8.3 – Comparison of MAE, RMSE and R² (%) across Models

Observations

- The two tree-based models clearly outperform Linear Regression and KNN, which shows that the relationship between the inputs and delivery time is non-linear.
- Linear Regression is the weakest model (R² of 0.5758), confirming that a purely linear model cannot capture the interactions between traffic, weather, distance and the other features.
- On this test split, the Decision Tree recorded the lowest errors and the highest R², with Gradient Boosting second.

9. Model Selection

Final model: Gradient Boosting Regression. It reached an MAE of 3.67 minutes, an RMSE of 4.61 minutes and an R^2 of 0.759, so it explains about 75.9% of the variance in delivery time and its predictions are typically within about 4 minutes of the actual time.

The Decision Tree scored slightly better on this single train-test split. Gradient Boosting was retained as the final model because an ensemble of many shallow trees is generally more stable on new data than a single tree, whose result depends heavily on how one particular split of the data falls. The gap between the two (about 0.4 minutes of RMSE) comes from one split without cross-validation, so it is not conclusive. Cross-validating both models is listed under future work.

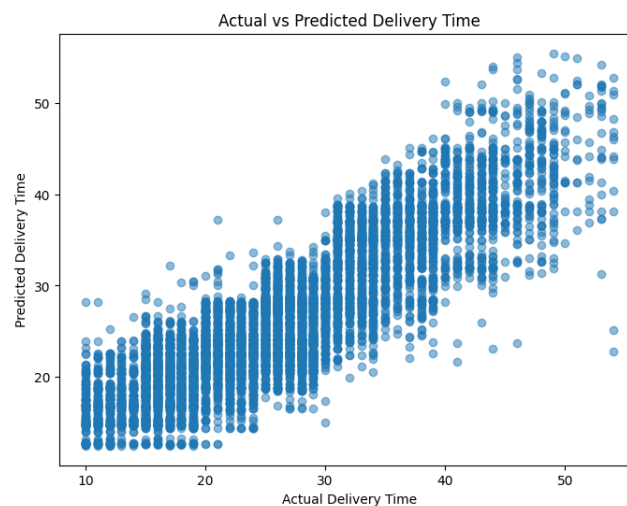


Figure 9.1– Actual vs Predicted Delivery Time (Gradient Boosting)

10. Implementation

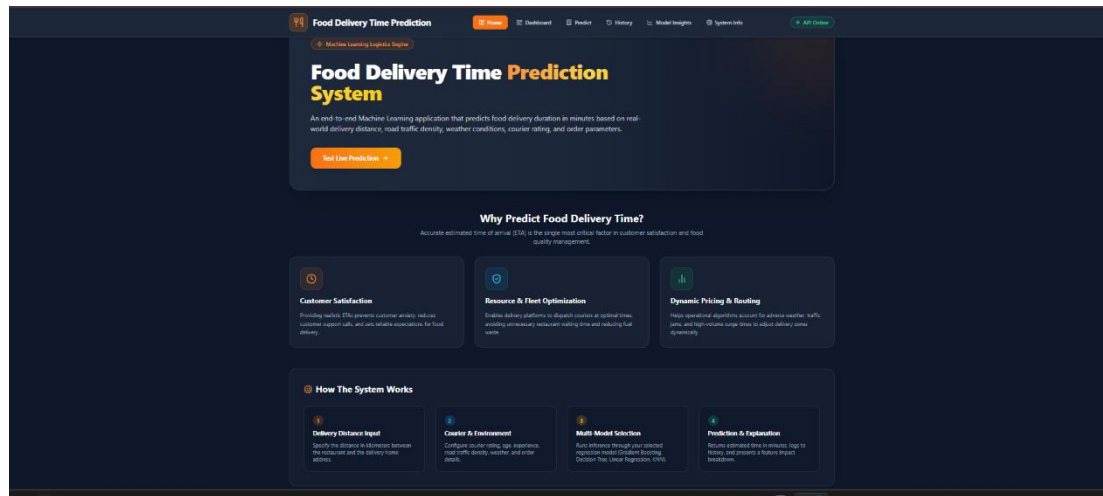


Figure 10.1– Home

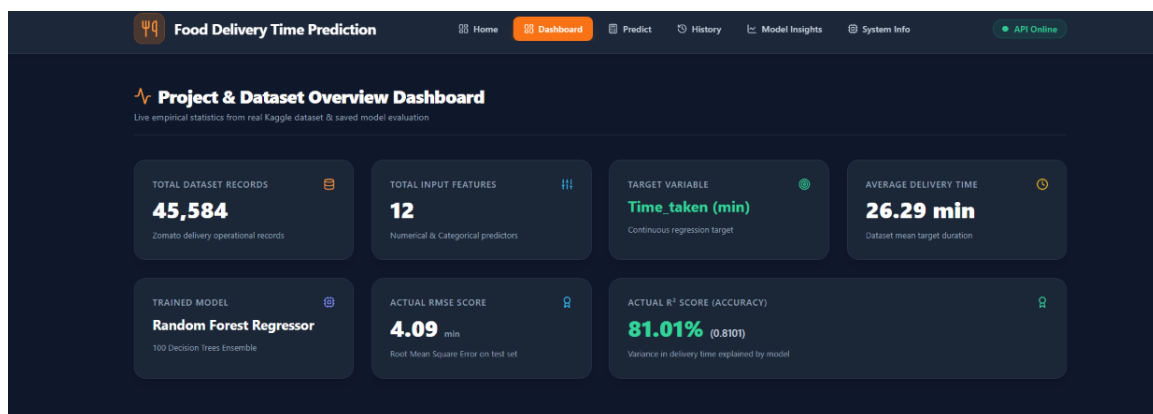


Figure 10.2 –Dashboard

The screenshot shows the 'Predict' page of the 'Food Delivery Time Prediction System'. The header includes navigation links: Home, Dashboard, Predict, History, Model Insights, System Info, and API Online. The main content area features a form titled 'Food Delivery Time Prediction' with instructions: 'Enter delivery distance between restaurant & home, specify environmental factors, then click Predict.' The form is divided into several sections: 'Distance Between Restaurant and Home' (Delivery Distance in Kilometers: 8.5 km), 'Courier Details' (Age: 28, Experience: 3.5, Rating: 4.8), 'Weather & Environment' (Weather: Sunny, Traffic Density: Medium, City Type: Metropolitan), and 'Vehicle & Order Details' (Vehicle Type: Motorcycle, Order Type: Meal, Vehicle Condition: 2 (Good), Multiple Deliveries: 1 Delivery, Festival Season?: No). A 'Predict Delivery Time' button and a 'Clear Form' button are at the bottom. A message on the right states: 'No Prediction Generated. Select your model, enter coordinates, then click Predict Delivery Time to view the ML prediction & evaluation metrics.'

Figure 10.3 –Predict

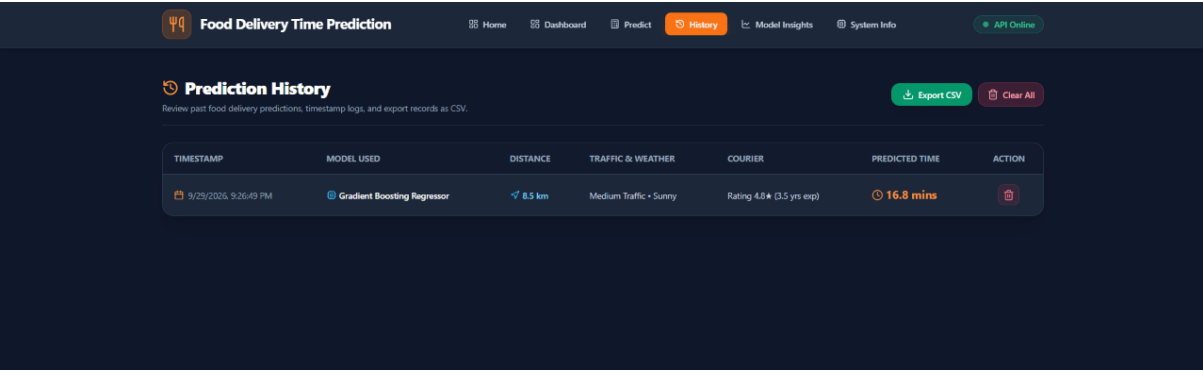


Figure 10.4–History

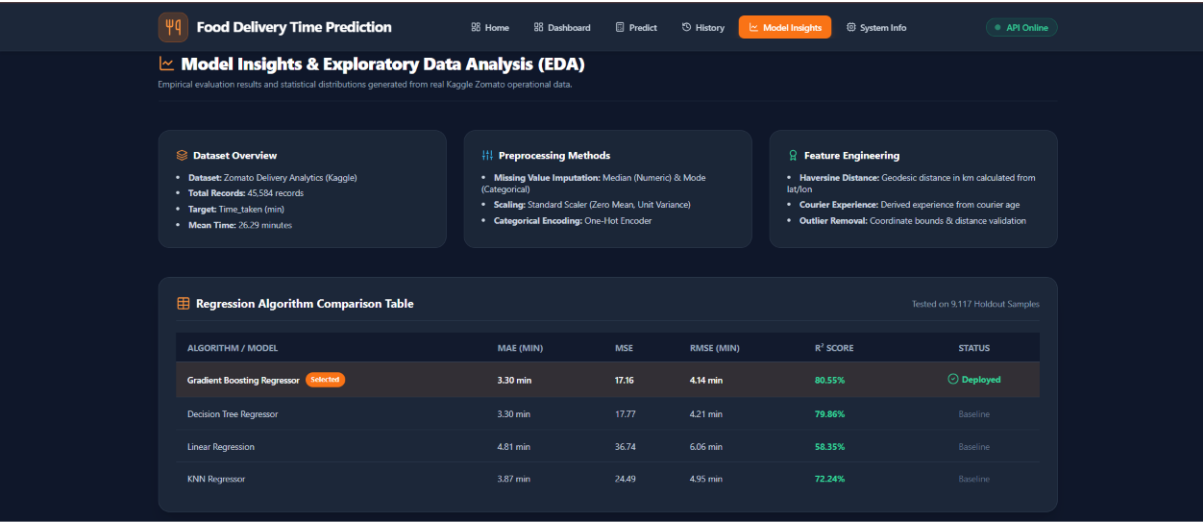


Figure 10.5 –Model Insights

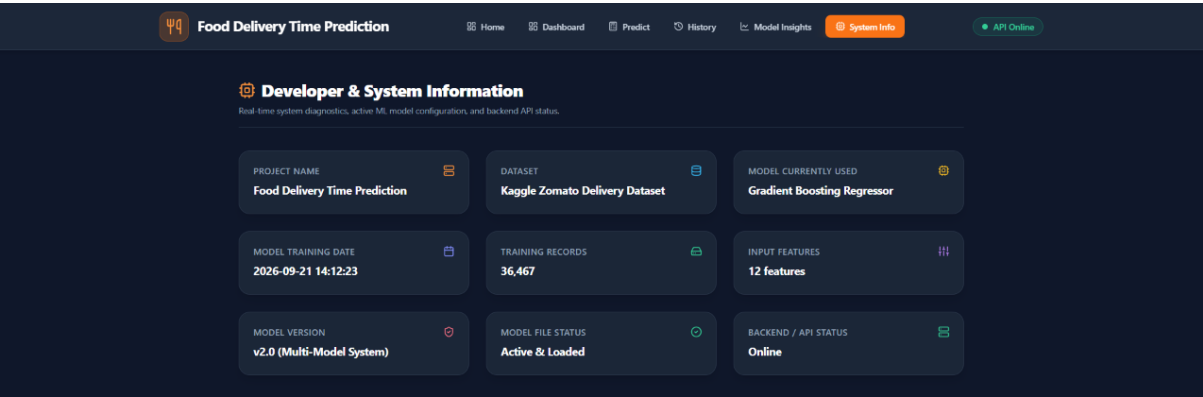


Figure 10.6 –System Info

11. Conclusion

This project built a machine learning pipeline to predict food delivery time from rider, order, traffic, weather and location information. After cleaning the data, engineering a Haversine distance feature and one-hot encoding the categorical variables, four regression models were trained and compared. Tree-based models performed best, and the final Gradient Boosting model predicts delivery time with an MAE of 3.67 minutes and an R^2 of 0.759. The model is saved as a reusable pipeline and can be used for delivery time estimation in a prediction application.

Future Scope

- **Real-Time Traffic Integration:** Use live traffic data to improve delivery-time predictions.
- **Weather Data Integration:** Include real-time weather conditions for more accurate predictions.
- **Larger and Diverse Datasets:** Train the model with larger datasets from different cities and regions.
- **Mobile Application:** Develop a mobile app for customers and delivery partners.
- **Restaurant Preparation Time:** Include real-time food preparation time as an input feature.
- **Delivery Partner Availability:** Consider the availability and workload of delivery partners.
- **Continuous Model Retraining:** Regularly update the model with new delivery data to maintain prediction accuracy.
- **Platform Integration:** Integrate the prediction system with food-delivery platforms for real-world usage.

12.References

Project GitHub Repository Links

24501A05M0- <https://github.com/Jithamanyu0210/FoodDeliveryTime>

24501A05P6- <https://github.com/Vaishnavi-2728/FoodDeliveryTime>

24501A05L5- <https://github.com/nidhishaw0888-creator/FoodDeliveryTime>

24501A05N6-<https://github.com/thummalasruthi954/FoodDeliveryTime>

Dataset Link from Kaggle Website

<https://www.kaggle.com/datasets/saurabhbadole/zomato-delivery-operations-analytics-dataset>

Scikit-Learn Documentation

<https://scikit-learn.org/stable/>

Python Documentation

<https://docs.python.org/3/>

Pandas Documentation

<https://pandas.pydata.org/docs/>

NumPy Documentation

<https://numpy.org/doc/>

Matplotlib Documentation

<https://matplotlib.org/stable/>

Seaborn Documentation

<https://seaborn.pydata.org/>